University of Nottingham
UK | CHINA | MALAYSIA

Developing Maintainable Software

COMP2013 – Lecture 02

Marjahan Begum and Max L. Wilson

Today's Lecture Slot

Introduction to DevOps

30mins

Introduction Continuous Integration and Continuous Deliver(CI/CD)

30mins

Plan for Lab 2

20mins

Learning Outcomes for this Week

- At the end of this week …
  - Understand what is DevOps in terms of principles
  - Be able to understand and apply role of business and people in the organisation to create values through Dev and Ops
  - Understand how to operationalize DevOps in a software companies through CI/CD
  - Practice some of the DevOps principles through different scenarios

Reading this week

- Read Chapter's 1 and 2 in The DevOps Handbook – How to Create World-Class Agility, Reliability and Security in Technology Organisations by Gene Kim, Jez Humble, Patric Debois and John Willis.
- This is available in the library: https://rl.talis.com/3/notts/lists/EDCA0F02-AE7C-304A-7596-DDEB8806EF3A.html?lang=en-GB

Timetable of Activities

W	W/c	Lecture - Mon 1-3pm	Lecture includes...	Lab Wed 10-1pm
1	29th Sept	Module Introduction	Guest Interview (confirmed)	Refreshing your OO Skills
2	6th Oct	Intro to DevOps	Industry Guest (TBC)	DevOps Scenario Comparisons
3	13th Oct	Refactoring: Identifying Bad Code (Dev)		Refactoring Exercises 1
4	20th Oct	Refactoring: Building GUIs (Dev)	CW Release	Refactoring Exercises 2
5	27th Oct	Refactoring: Design Patterns (Dev)		Refactoring Exercises 3
6	3rd Nov	Refactoring: Software Architecture (Dev)		Refactoring: Reviews
7	10th Nov	Developer's Tools		Refactoring Exercises 4 (TBC)
8	17th Nov	DevOps Infrastructure as Code	Industry Guest (TBC)	Docker Exercise
9	24th Nov	DevOps Infrastructure as Code	Guest Live Demo (TBC)	CI/CD Pipeline Exercise
10	1st Dec	DevOps Deployment		Coursework Support
11	8th Dec	DevOps Optimisation	Final CW Q&A	Coursework Support

Dev

Ops

NOTE: some aspects of the timetable are subject to change.

Introduction to DevOps

Job skills (Blackbaud)

Build automation leveraging CI/CD processes, automated testing, unit testing, code coverage and other software development best practices in both public and private cloud technologies

Job skills (Deloitte)

- Create, implement, and manage the development of scripts to automate everyday operations
- Monitor and maintain multiple (DevOps) environments based on client requirements
- Lead program optimization to work within constructs of an AWS environment
- Manage system migrations and system upgrades to create and deploy new cloud environments

Job skills (Rakuten)

- Automate deployments for new services
- Provision infrastructure using Amazon Web Services
- Investigate performance issues
- Assist developers with build and release tasks
- Promote best practices for information security

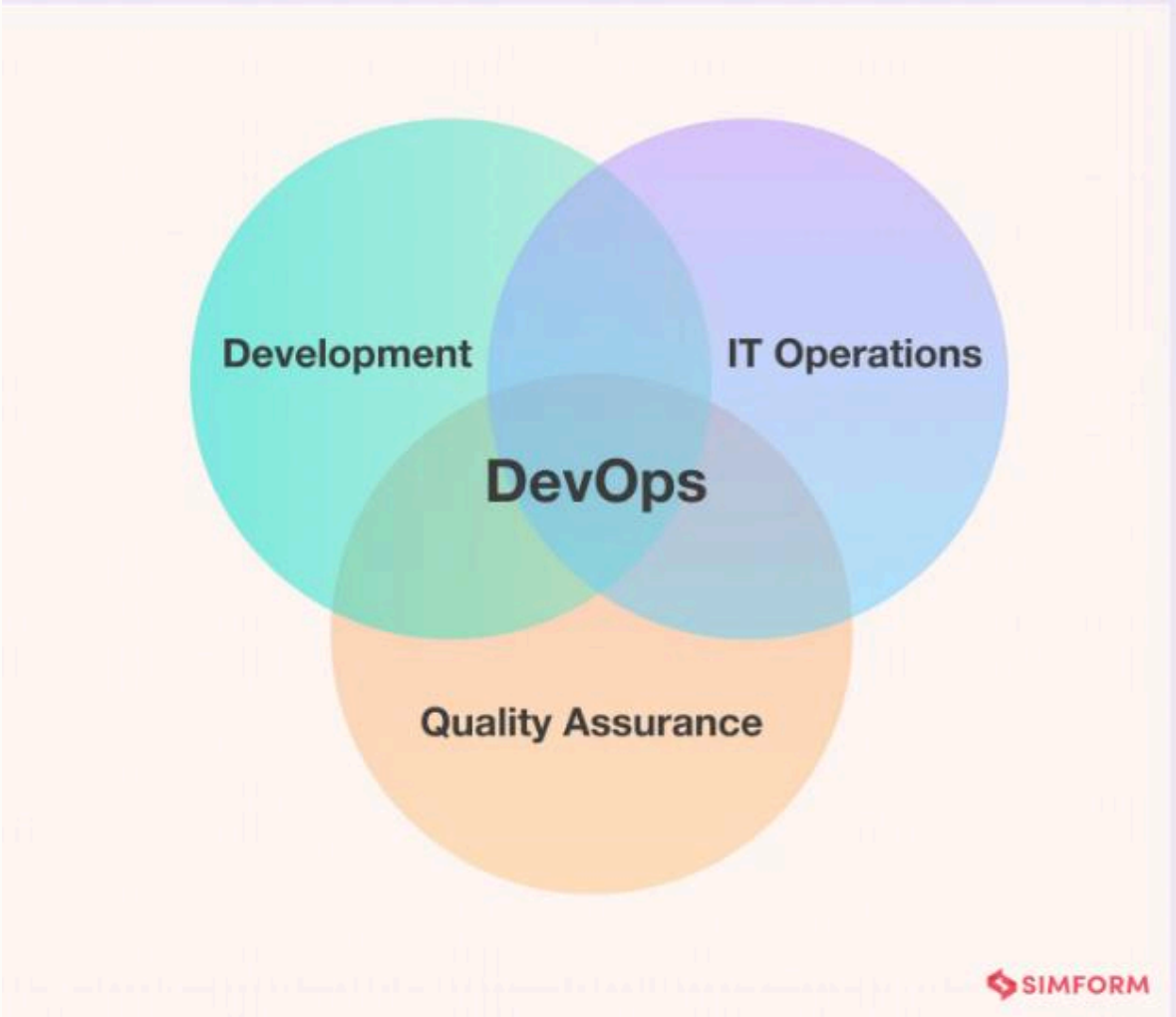
History of DevOps

- Several IT “movements” in the last 20 years
- Lean movement from manufacturing
- Agile Movement
- Agile Infrastructure movement
- CD – continuous delivery movement

What is DevOps?

"The practice of operations and development engineers participating together through the entire service lifecycle, from the design and development process all the way to production support"

What is DevOps



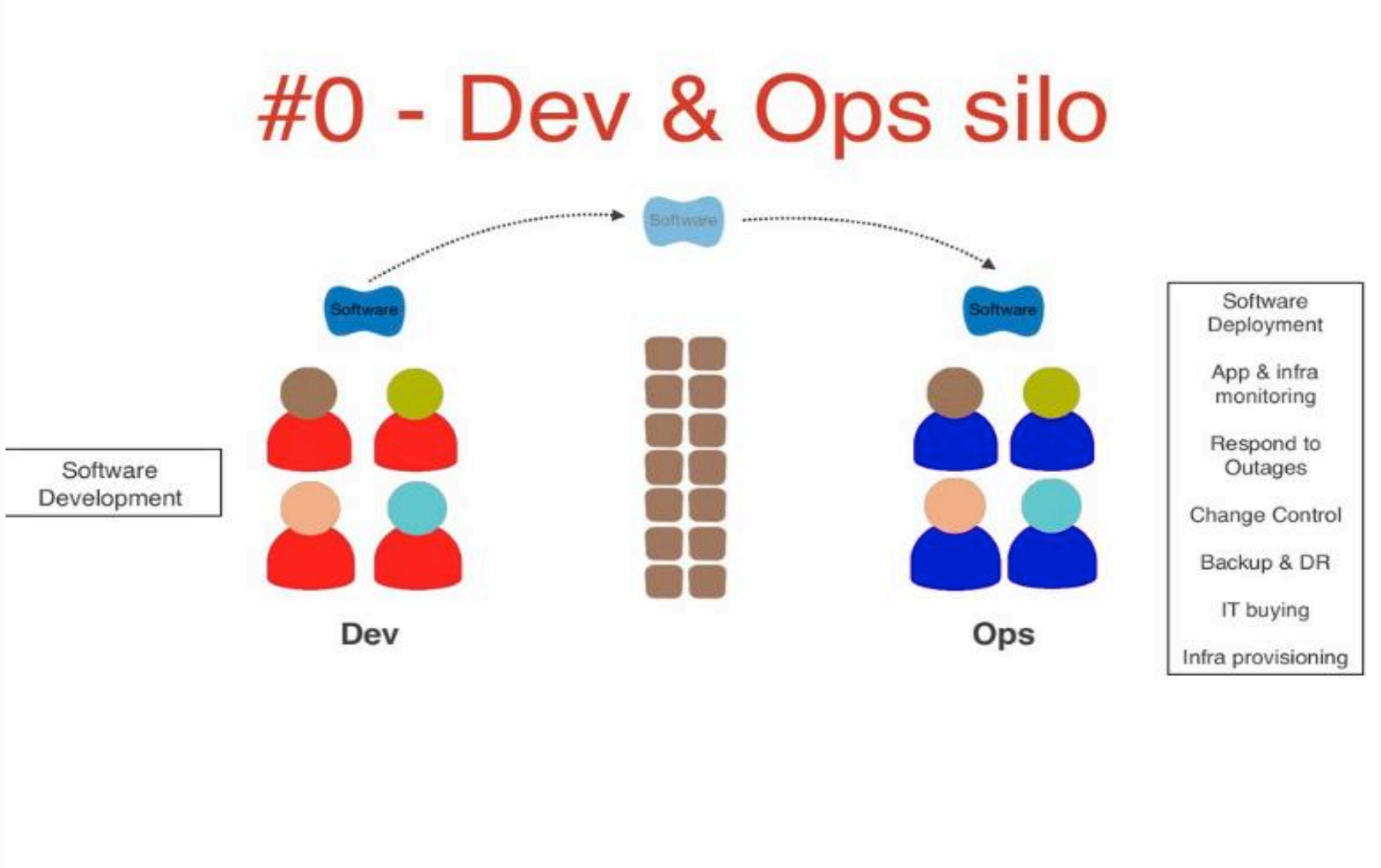
Why DevOps

- Effective in improving IT and business outcomes
- Elite team deploy 973 times more frequently with 6,570 times shorter lead time than low IT performers.
- Heightened productivity cuts burnout in half

What is DevOps – Core values

- Culture – Change people's behavior
- Automation – to accelerate change
- Measurement for improvement
  - Mean time to restore services
  - Cycle time to deploy a new features
  - Costs and revenues
  - Employer satisfaction
- Sharing
  - Teamwork, trust, and transparency, collaboration

Culture Breaking down the Silos



Conway’s law

”Organizations which design systems, are constrained to produce designs which ware copies of communication structures of these organization”

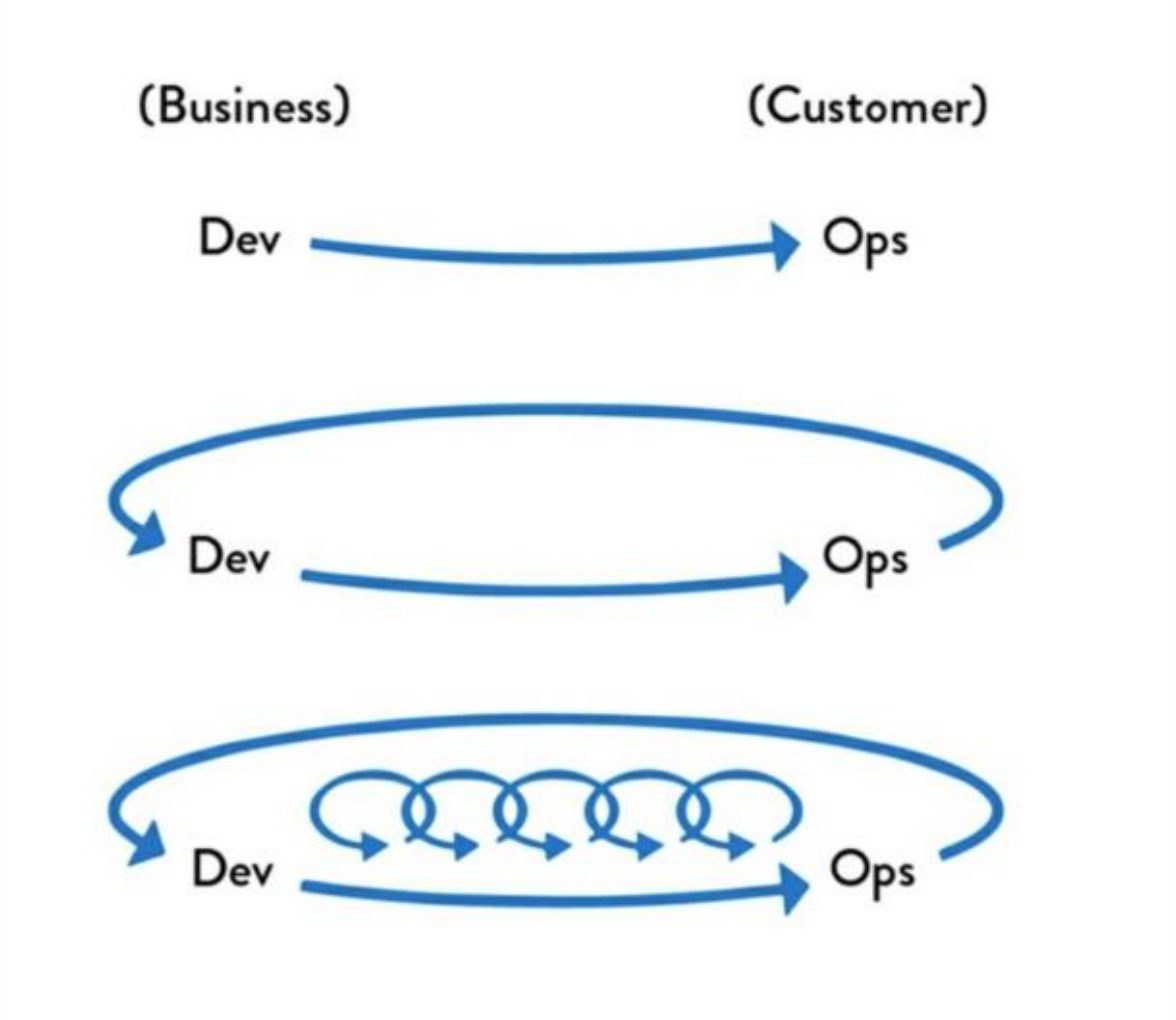
Melvin Conway (1960)

The Three Ways

1. Systems Thinking and the Principle of Flow

2. Systems Thinking and the Principle of Flow

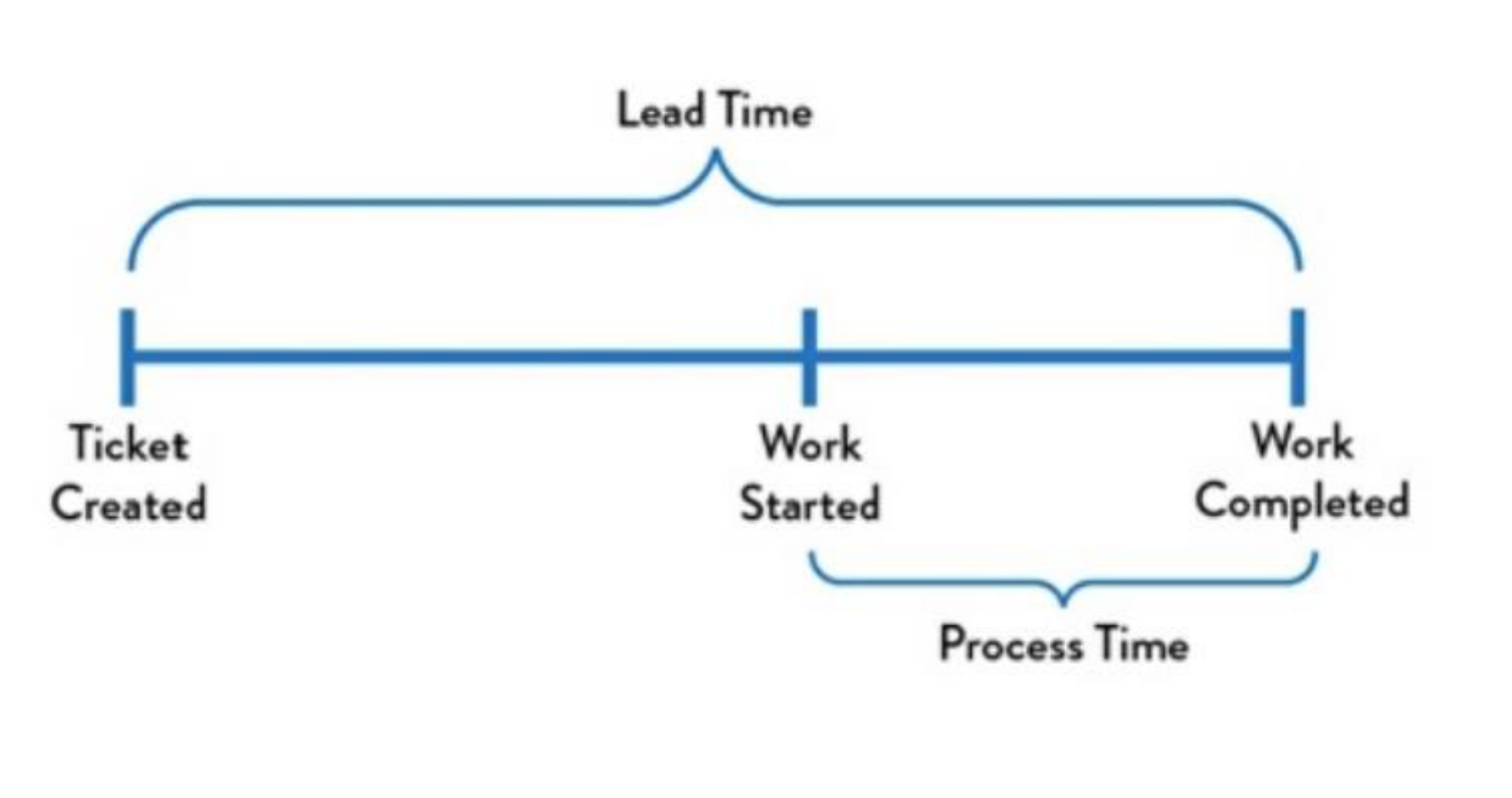
3. A Culture of Continuous Experiment and Learning



The First Way - Systems Thinking and the Principle of Flow

- Making work “visible”
- Limiting work-in-progress (WIP)
- Reducing batch sizes
- Reducing hands-off between teams
- Continually identify and elevate our constraints
- Identifying and removing constraints and waste.

Process Time vs. Lead time



The trend toward faster, cheaper, low-risk delivery of software

	1970s–1980s	1990s	2000s–Present
Era	Mainframes	Client/Server	Commoditization and Cloud
Representative technology of era	COBOL, DB2 on MVS, etc.	C++, Oracle, Solaris, etc.	Java, MySQL, Red Hat, Ruby on Rails, PHP, etc.
Cycle time	1–5 years	3–12 months	2–12 weeks
Cost	$1M–$100M	$100k–$10M	$10k–$1M
At risk	The whole company	A product line or division	A product feature
Cost of failure	Bankruptcy, sell the company, massive layoffs	Revenue miss, CIO's job	Negligible

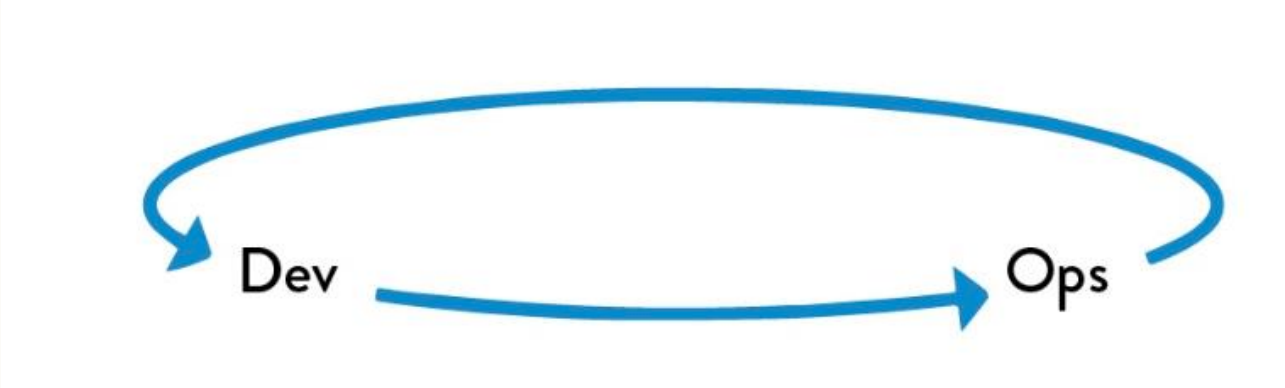
(Source: Adrian Cockcroft, "Velocity and Volume (or Speed Wins)," presentation at FlowCon, San Francisco, CA, November 2013.)

Organizational Types

Pathological	Bureaucratic	Generative
Information is hidden	Information may be ignored	Information is actively sought
Messengers are "shot"	Messengers are tolerated	Messengers are trained
Responsibilities are shirked	Responsibilities are compartmented	Responsibilities are shared
Bridging between teams is discouraged	Bridging between teams is allowed but discouraged	Bridging between teams is rewarded
Failure is covered up	Organization is just and merciful	Failure causes inquiry
New ideas are crushed	New ideas create problems	New ideas are welcomed

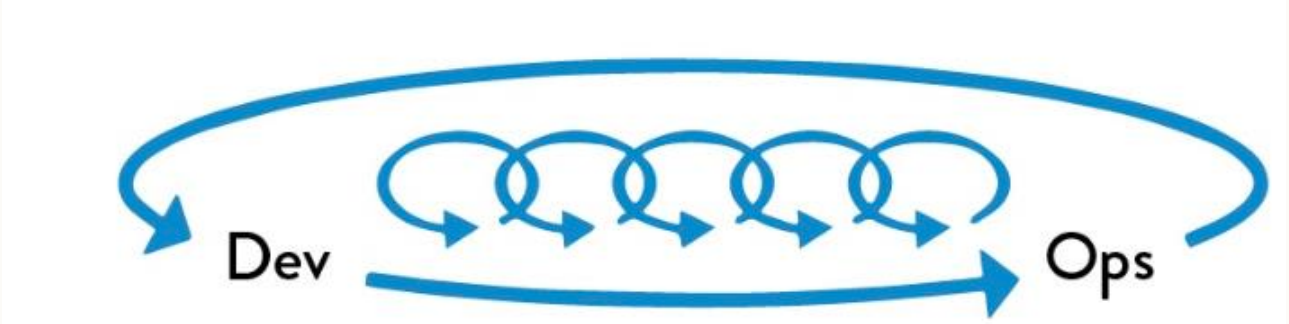
The Second Way – Amplify Feedback

- Enables fast and constant flow of feedback from right to left at all stages of our value stream
- Requires that we amplify feedback to prevent problems from happening again or enable faster detection of recovery
- We create quality at the source and generate or embed knowledge where it is needed
- We see problems as they occur and swarm theme until effective counter measures are in place



The Third Way – Continuous Learning and Experimentation

- High-trust culture
- Creation of individual knowledge
- Organizational knowledge
- Leaders reinforce a learning culture
- Inject production failure to create resilience



Lesson from Google

- Data-driven decision
- Transparency
- Shared goals
- No-blame policy
- Education

Stage of Software Development

- Build and test code
- Deploy and test software
- Release software or service

Introduction to Continuous Integration and Continuous Deployment

Continuous Integration

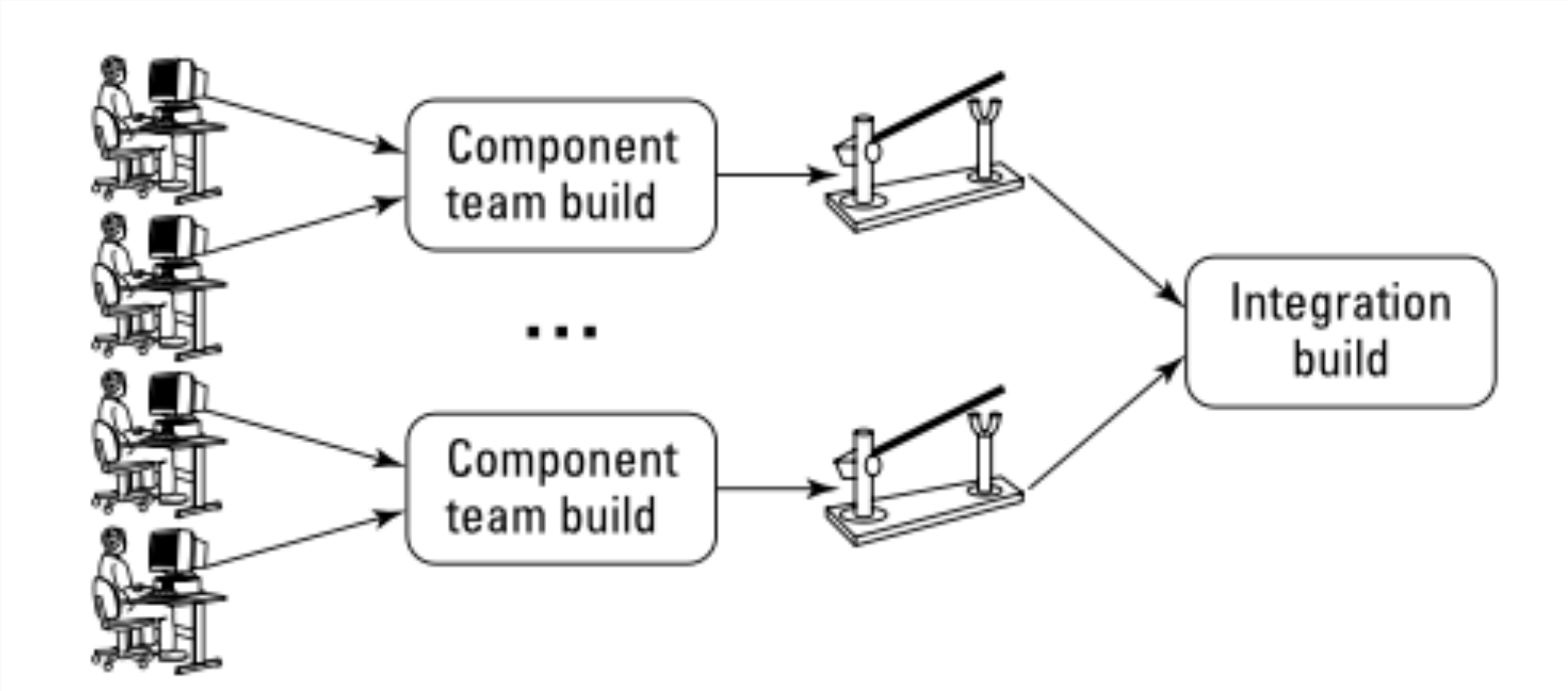
Continuous Integration is a development practice that requires developers to integrate code into a shared repository several times a day. Each check-in is then verified by an automated build, allowing teams to detect problems early on.

Continuous Integration doesn’t get rid of bugs, but it does make them dramatically easier to find and remove. – Martin Fowler

Why Continuous Integration

- Differences between Dev and production environment
- Cross-browser or cross-version testing
- Hard to protect against security threats
- Stress testing/longevity testing of new feature/code paths
- Example: Salesforce CI runs 150K + tests and automatically opesn bug reports when test fails.

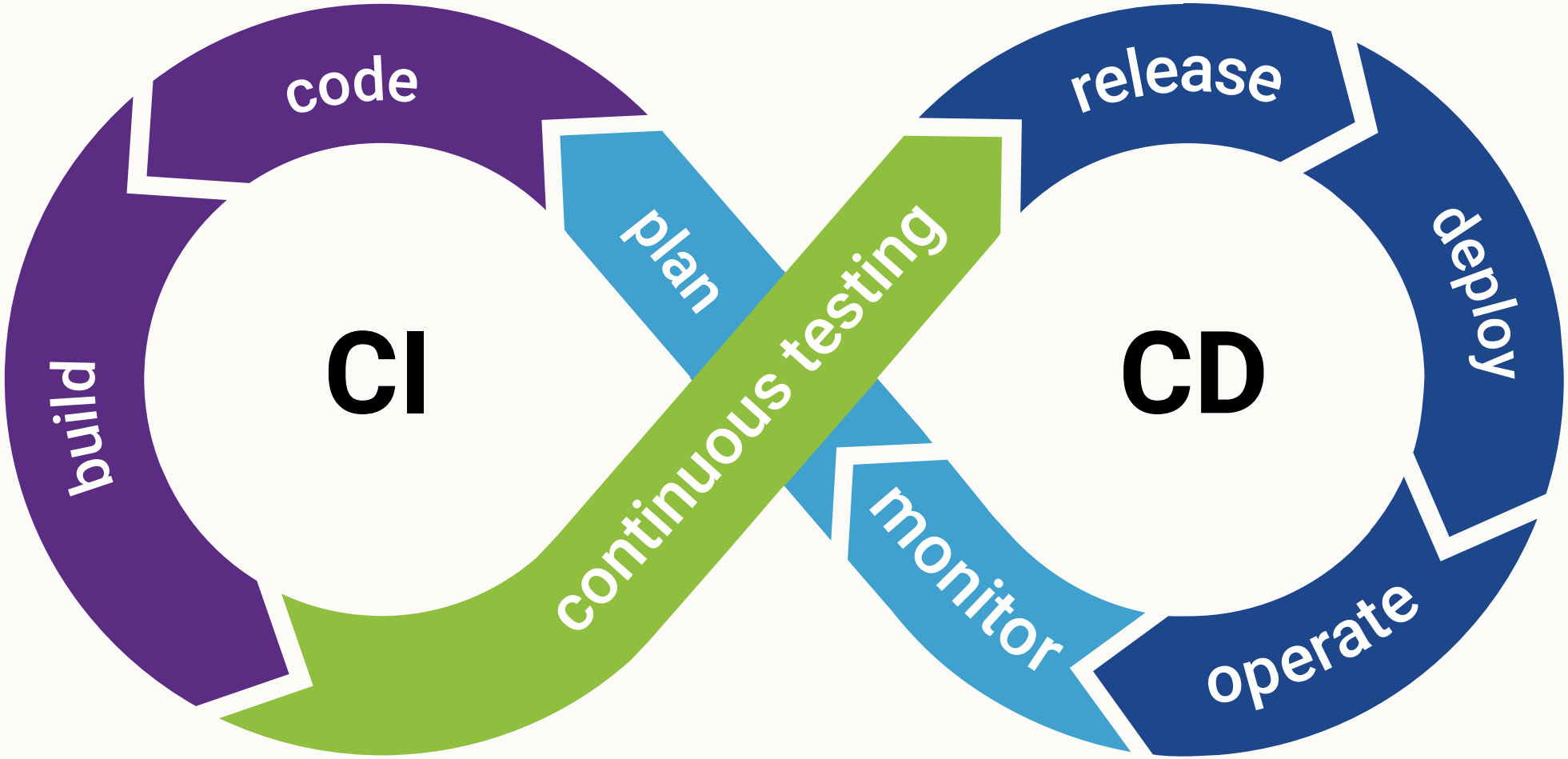
Collaborative Development



Continuous Delivery

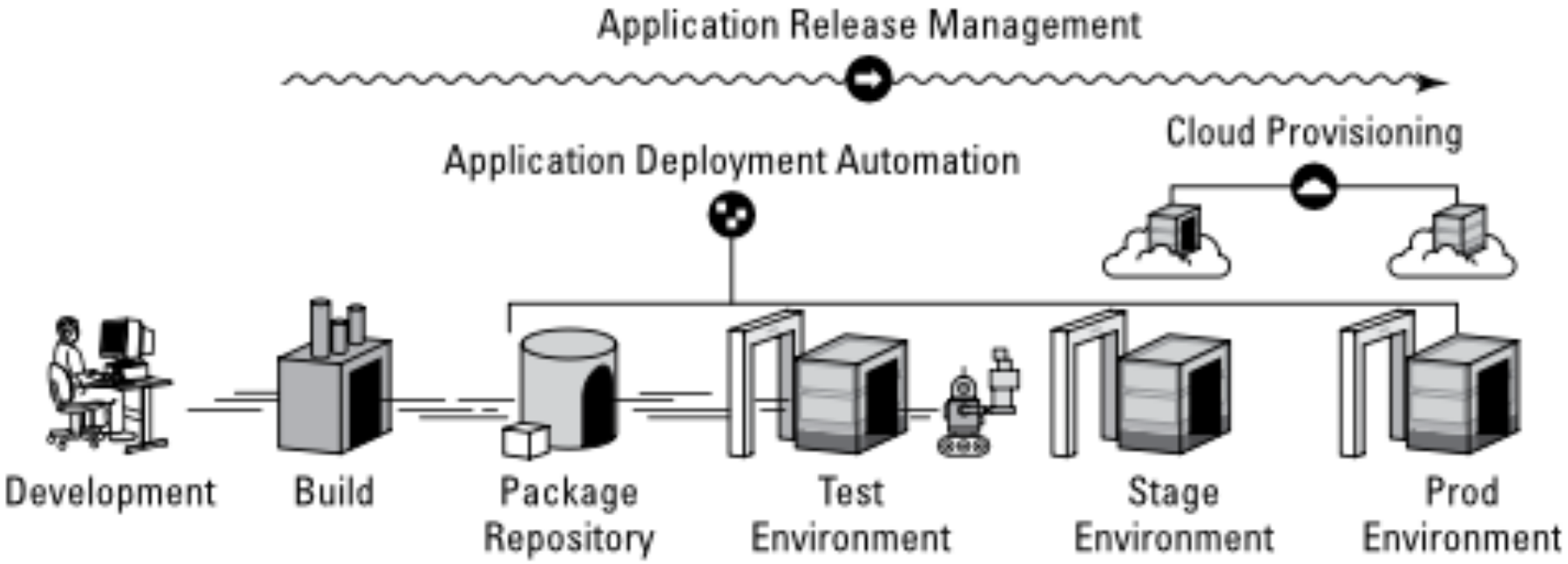
The ability to get changes – features, configuration changes, bug fixes, experiments - into production or into the hands of users safely and quickly in a suitable way.

CI/CD

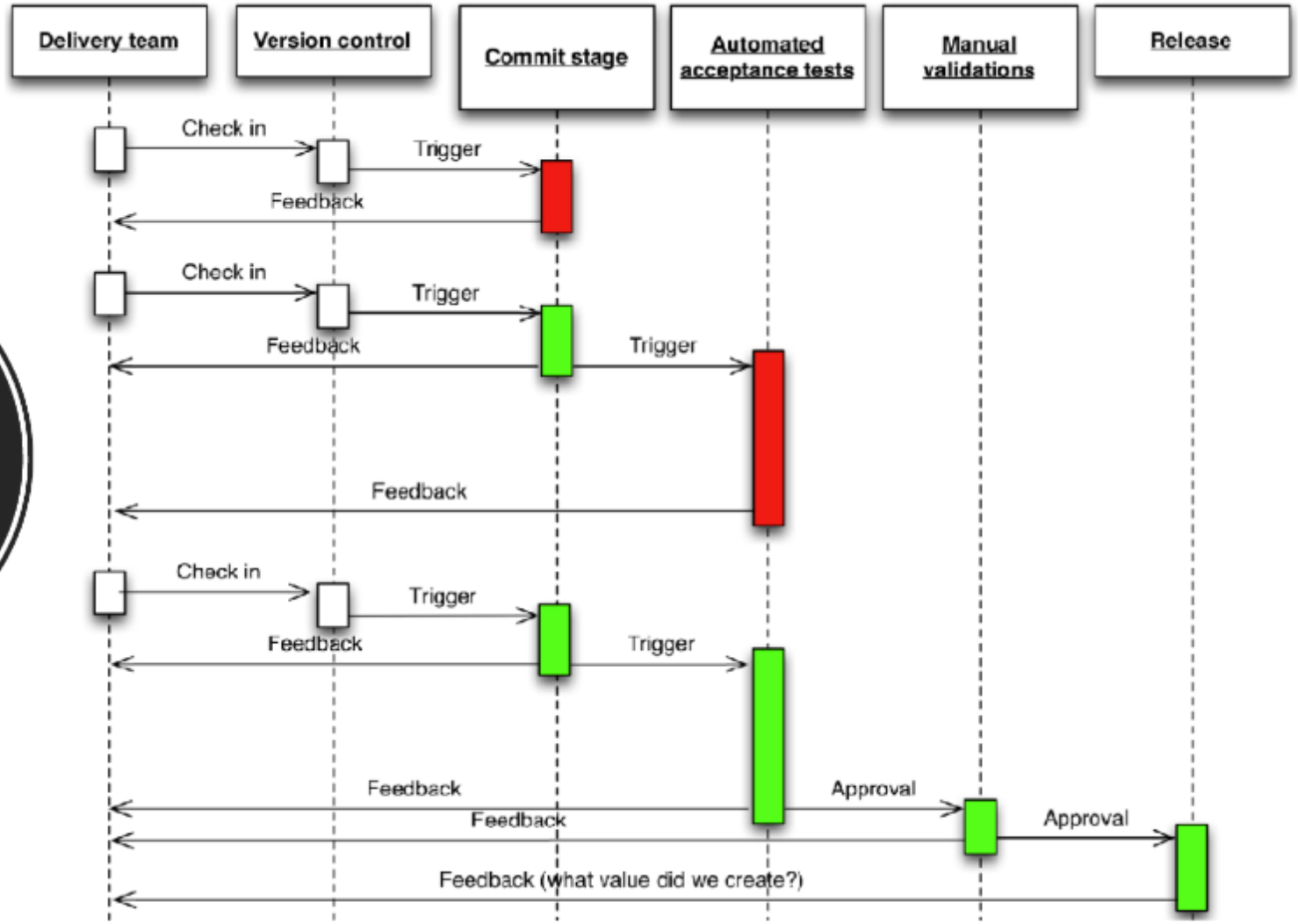

code
build
CI
plan
continuous testing
release
deploy
CD
monitor
operate

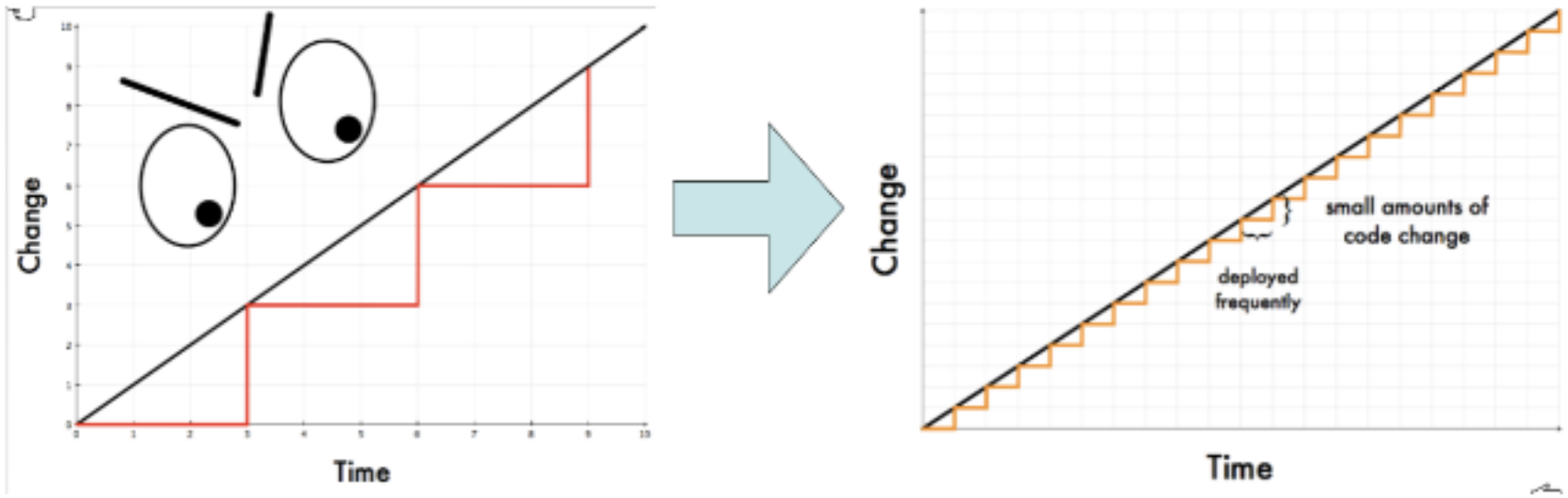
DevOps pipeline

DevOps pipeline



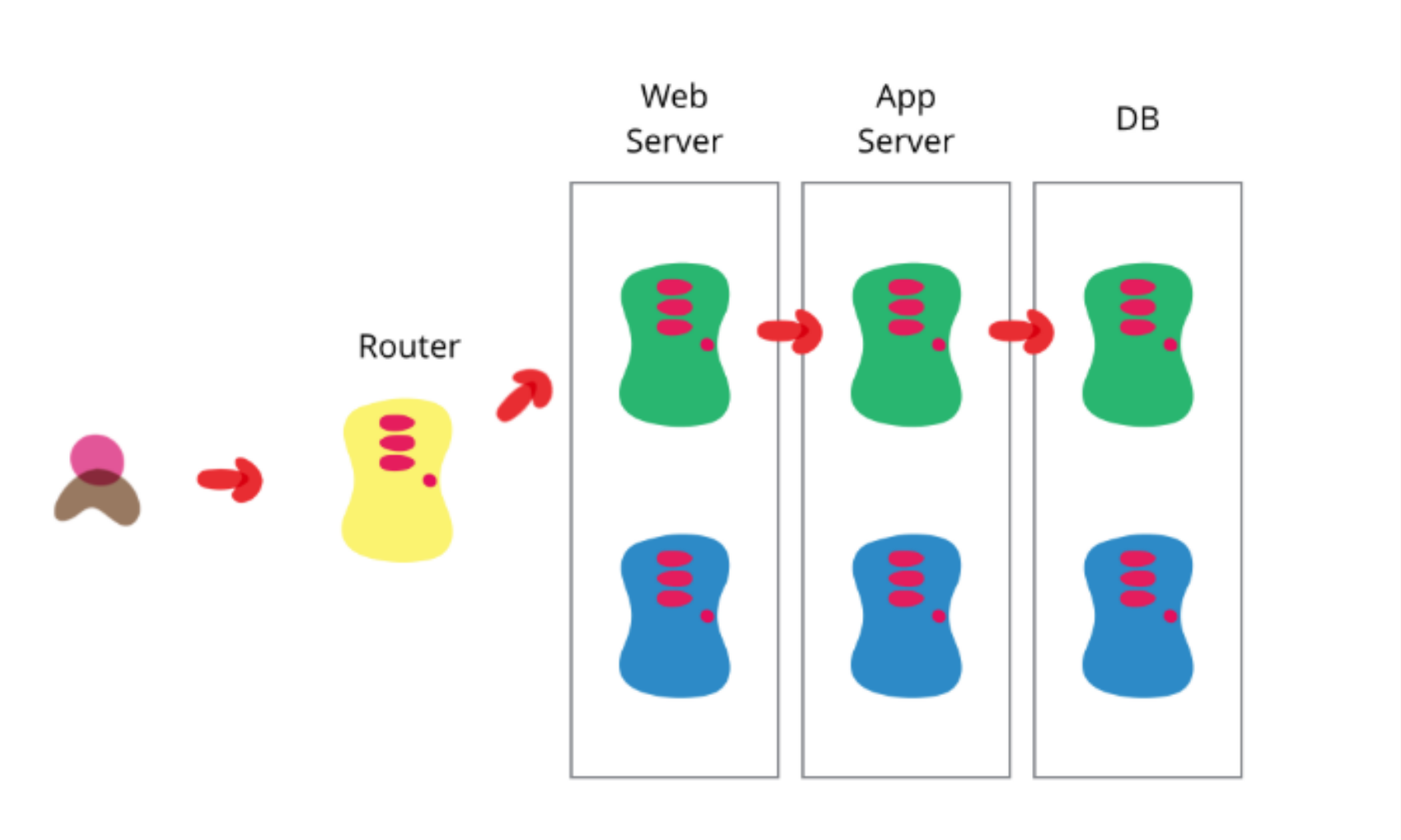
Deployment pipeline (Jez Humble)





John Allspaw: “Ops Metametrics” http://slidesha.re/dsSZIr

Blue Green Deployment

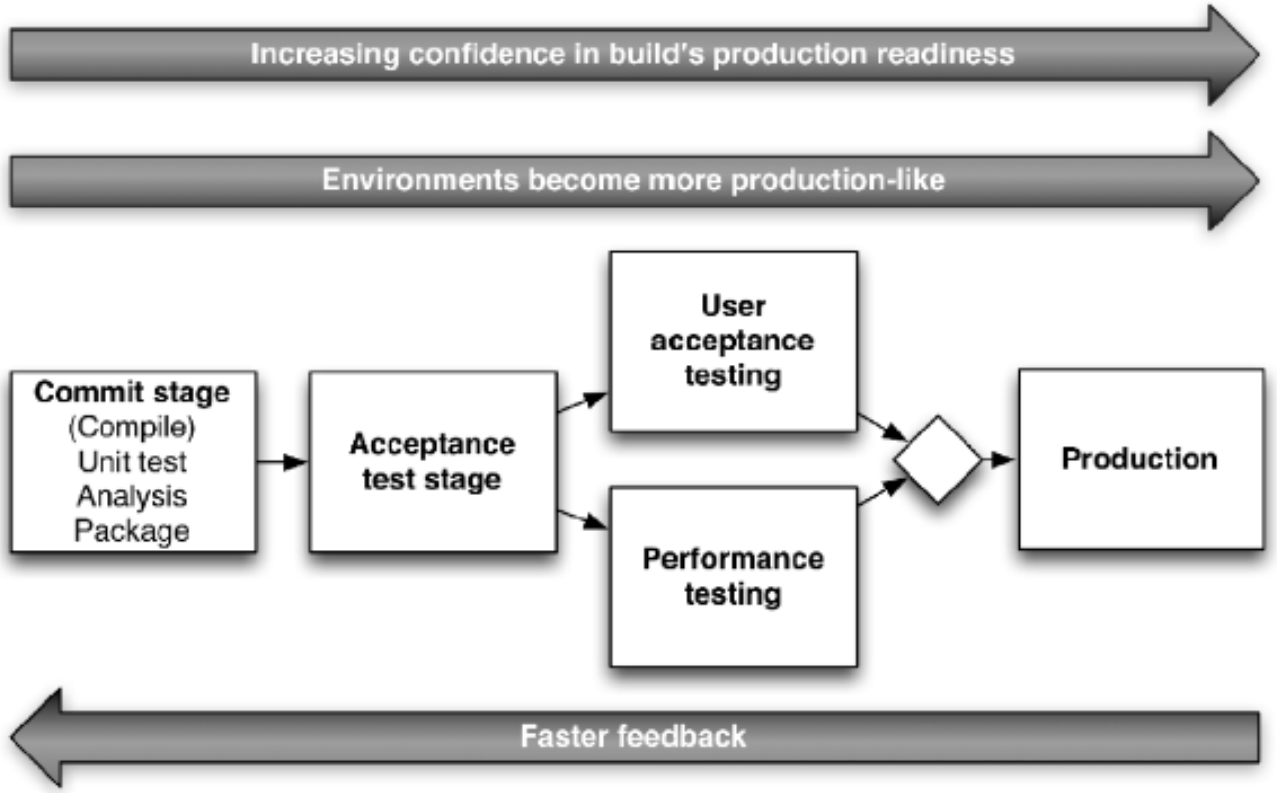


Martinflower.com

Benefits of CI/CD

- Automation of the software release process i.e. painless
- Lower testing costs because of multiple tests being run in seconds.
- Bug issues are quickly identified and resolved.
- The release time of the software is faster to reach the market through continuous testing and development.
- In cloud environments, Continuous Delivery allows teams to manage cloud resources effectively.
- Improved accessibility and communication in teams.
- Increased customer and employee satisfaction

Trade offs


Increasing confidence in build's production readiness
Environments become more production-like
Commit stage
(Compile)
Unit test
Analysis
Package
Acceptance test stage
User acceptance testing
Performance testing
Production
Faster feedback

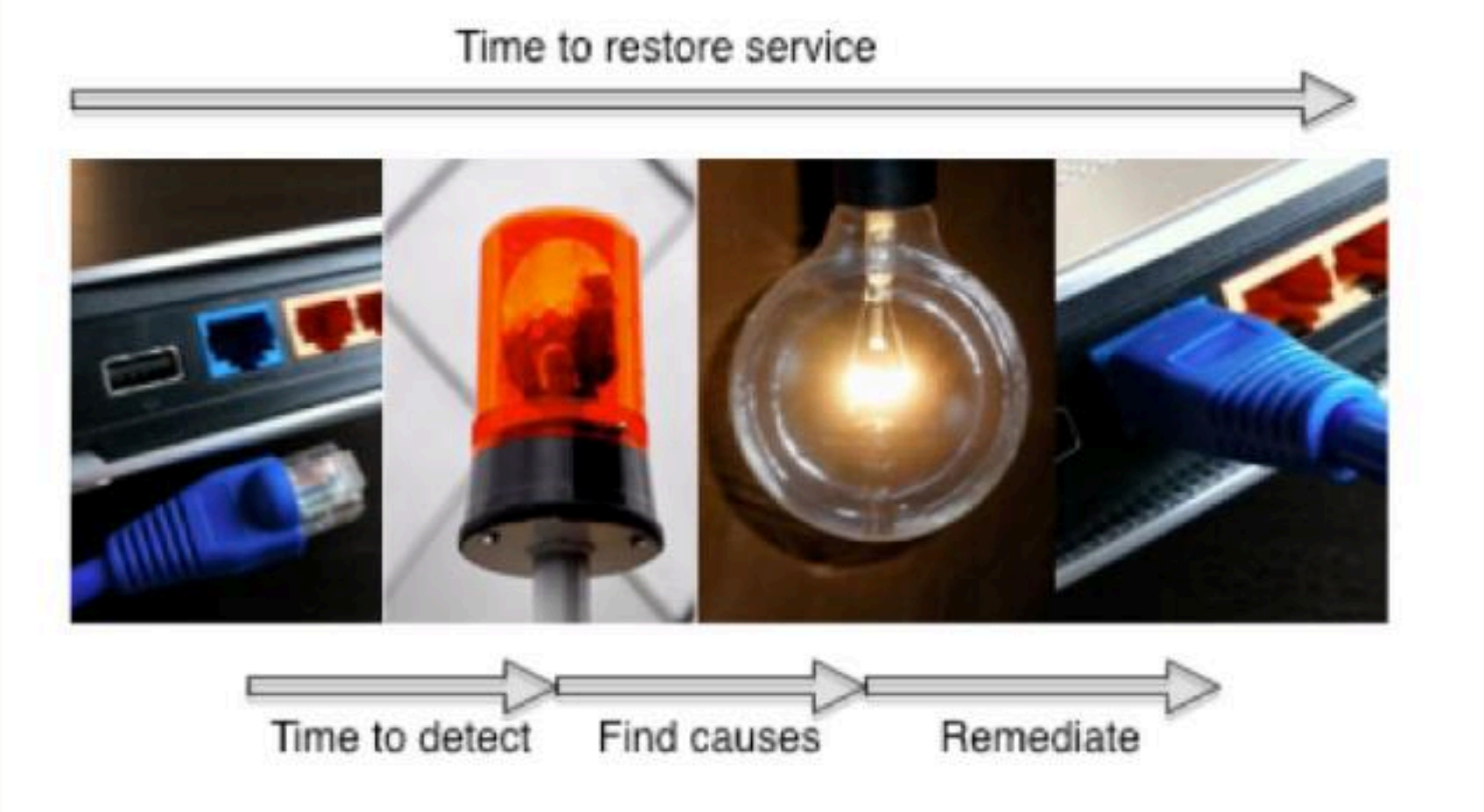
Key high level measurements to improve CI/CD

- Deployment frequency
- Lead time for change
- Meant time to recover
- Change of failure rate

Key high level measurements to improve CI/CD

- Deployment frequency - Automated builds and testing are a need for automated deployment.
- Lead time for change - Build/test automation and early feedback assist in reducing the amount of time it takes to get code from commit to successfully operating in production.
- Mean time to recover – Average time it takes to restore from failure of the system.
- Change of failure rate - The number of faults that reach production is substantially decreased by early automated testing.

Optimize for Mean Time Recover (MTR)



Software Engineering Benchmarks By Org Size

SCALE-UP
200-1000 Employees

3,026 Orgs | 6,100,878 Pull Requests | 167,437 Active Contributors |
Time metrics in minutes or hours, as noted

Category	Metric	Elite	Good	Fair	Needs Focus
DevEx	Coding Time (hours)	< 3	3 - 6	7 - 28	> 28
	Pickup Time (mins/hours)	< 65 mins	65 mins - 5 hours	6 - 17	> 17
	Approve Time (hours)	< 15	15 - 23	24 - 44	> 44
	Merge Time (mins/hours)	< 75 mins	75 mins - 4 hours	5 - 18	> 18
	Review Time (mins/hours)	< 21 mins	21 - 56 mins	57 mins - 3 hours	> 3
	Deploy Time (hours)	< 16	16 - 103	104 - 265	> 265
	Merge Frequency (per dev/week)	> 1.85	1.85 - 1.05	1.04 - 0.6	< 0.6
	PR Maturity (%)	> 90%	90 - 83%	82 - 76%	< 76%
DORA	Cycle Time (hours)	< 47	47 - 93	94 - 172	> 172
	Deploy Frequency (per service)	> 0.8	0.8 - 0.4	0.39 - 0.15	< 0.15
	Change Failure Rate (%)	< 1%	1 - 3%	4 - 15%	> 15%
	MTTR (hours)	< 8	8 - 12	13 - 35	> 35
Predictability	PR Size (code changes)	< 89	89 - 135	136 - 197	> 197
	Refactor Rate (%)	< 12%	12 - 16%	17 - 21%	> 21%
	Rework Rate (%)	< 4%	4 - 5%	6 - 7%	> 7%

What iss next

Executive summary

Our survey of more than 5,000 DevSecOps professionals worldwide showed that organizations are prioritizing investments in security, AI, and automatio
— and all of these are having positive effects on the experiences of developers and software engineering teams. However, this year's survey also
highlighted specific areas, such as software supply chain security, that warrant particular attention as organizations build out their DevSecOps strategies

AI is a core part of software development

78%

of respondents said they are currently using AI in software development or plan to in the next 2 years, up from 64% in 2023

Organizations are serious about automation

67%

of respondents said their software development lifecycle is mostly or completely automated

The toolchain struggle is real

64%

of respondents said they want to consolidate their toolchain

Software supply chain security is key

67%

of developers said a quarter or more of the code they work on is from open source libraries — but only 21% of organizations are currently using a software bill of materials (SBOM) to document the ingredients that make up their software components

Top IT investment priorities in 2024

1. Security
2. AI
3. DevSecOps platform
4. Automation
5. Cloud computing

Learn about the GitLab DevSecOps

A few good quotes…

- ”The bottleneck of today is no longer just technical practices (though they still exist); the biggest challenge and necessity is getting business leadership on board”

- ”Transformation must be co-created between the business and technology...”

- ”We must achieve true collaboration. 90% of that work involves getting the right people engaged, onboard, and aligned”

Timetable of Activities

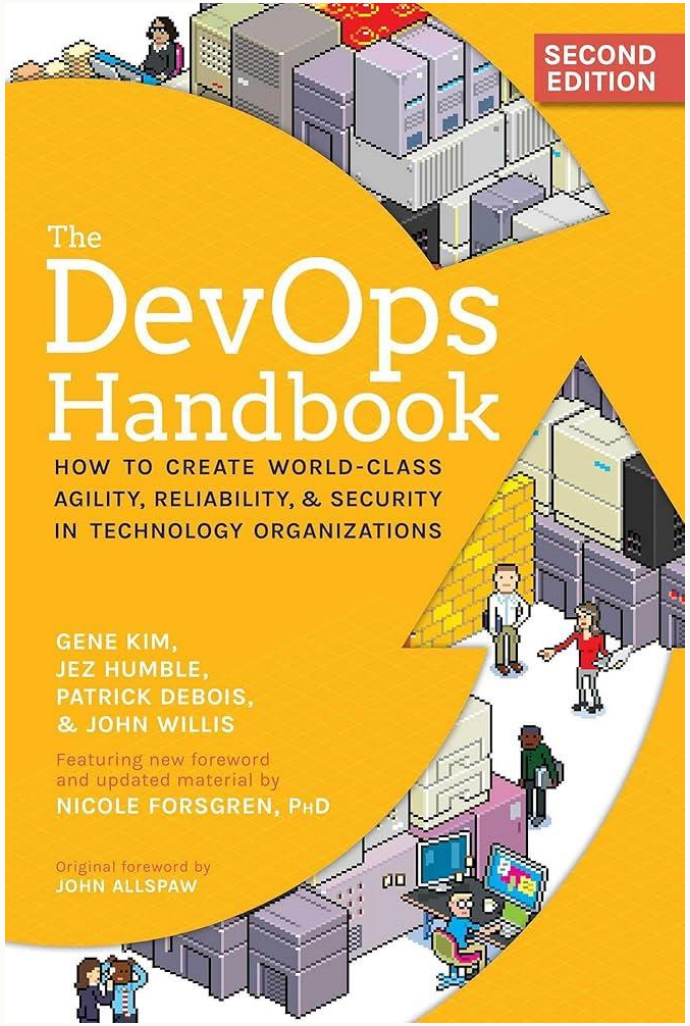
W	W/c	Lecture - Mon 1-3pm	Lecture includes...	Lab Wed 10-1pm
1	29th Sept	Module Introduction	Guest Interview (confirmed)	Refreshing your OO Skills
2	6th Oct	Intro to DevOps	Industry Guest (TBC)	DevOps Scenario Comparisons
3	13th Oct	Refactoring: Identifying Bad Code (Dev)		Refactoring Exercises 1
4	20th Oct	Refactoring: Building GUIs (Dev)	CW Release	Refactoring Exercises 2
5	27th Oct	Refactoring: Design Patterns (Dev)		Refactoring Exercises 3
6	3rd Nov	Refactoring: Software Architecture (Dev)		Refactoring: Reviews
7	10th Nov	Developer's Tools		Refactoring Exercises 4 (TBC)
8	17th Nov	DevOps Infrastructure as Code	Industry Guest (TBC)	Docker Exercise
9	24th Nov	DevOps Infrastructure as Code	Guest Live Demo (TBC)	CI/CD Pipeline Exercise
10	1st Dec	DevOps Deployment		Coursework Support
11	8th Dec	DevOps Optimisation	Final CW Q&A	Coursework Support

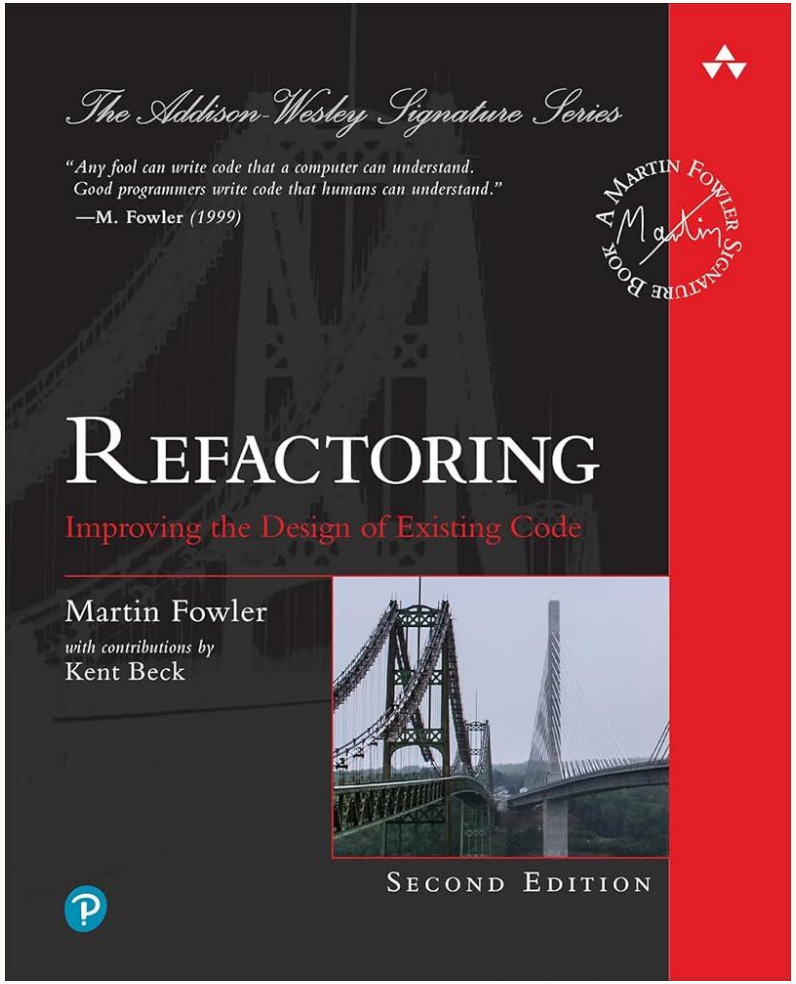
Dev

Ops

NOTE: some aspects of the timetable are subject to change.

Textbooks


SECOND EDITION
The
DevOps
Handbook
HOW TO CREATE WORLD-CLASS AGILITY, RELIABILITY, & SECURITY IN TECHNOLOGY ORGANIZATIONS
GENE KIM,
JEZ HUMBLE,
PATRICK DEBOIS,
& JOHN WILLIS
Featuring new foreword and updated material by
NICOLE FORSGREN, PHD
Original foreword by
JOHN ALLSPAW


The Addison-Wesley Signature Series
"Any fool can write code that a computer can understand.
Good programmers write code that humans can understand."
—M. Fowler (1999)
A MARTIN FOWLER SIGNATURE BOOK
Martin
REFACTORING
Improving the Design of Existing Code
Martin Fowler
with contributions by
Kent Beck
SECOND EDITION

Lab 02

- Read Chapter 1, 2 and 3 from The DevOps Handbook

- Practice strategies for implementing principles of DevOps.

Learning Outcomes for this Week

- At the end of this week …
  - Understand what is DevOps in terms of principles
  - Be able to apply role of business value and people in the organisation to create values through Dev and Ops
  - Understand how to operationalize DevOps in a software companies through CI/CD
  - Practice some of the DevOps principles through different scenarios

Questions?

why?
how?
when?
HOW?
WHAT?
WHO?
WHY?
WHERE?
WHEN?
how?
who?
what?
where?
WHAT?
WHEN?
how?
HOW?

References and Acknowledgements

- The DevOps Handbook – How to Create World-Class Agility, Reliability and Security in Technology Organisations by Gene Kim, Jez Humble, Patric Debois and John Willis
- Martinflowler.com
- https://learn.gitlab.com/
- Michael Verdicchio, Department of Cyber and Computer Sciences The Citadel Charleston, SC – from his course on DevOps
- Check this online course out: https://www.coursera.org/learn/devops-culture-and-mindset?action=enroll